

Case Study: Personal Loan Application Processing System

1. Project overview

A financial services company wants a console-based application to manage personal loan applications.

The application must allow loan officers to create, review, approve, reject, disburse, and close loan applications. It must enforce business validations before information is stored in PostgreSQL.

The project must use:

- Spring Framework IoC container
- Spring annotations and Java configuration
- Standard JDBC
- PostgreSQL
- Lombok
- Maven
- A properties file for database configuration

The project must not use:

- Spring Boot
- Spring Data JPA
- Hibernate
- JPA
- JdbcTemplate
- REST APIs
- Multiple business entities

Only one business entity, `LoanApplication`, must be used.

2. Business problem

The company currently processes loan applications manually.

This causes problems such as:

- Duplicate applications
- Invalid applicant information
- Loan requests beyond permitted limits
- Applications approved without proper eligibility checks

- Incorrect interest rates
- Invalid status transitions
- Loans disbursed before approval
- Missing rejection reasons
- Applications created for underage applicants
- Loan tenures that exceed company policies

The new system must prevent these problems using application-level and database-level validations.

3. Main entity

LoanApplication

The `LoanApplication` entity represents a customer's request for a personal loan.

Suggested fields:

Field	Purpose
ID	Database-generated primary key
Application number	Unique loan application identifier
Applicant name	Full name of the applicant
Email	Applicant's email address
Phone number	Applicant's mobile number
Date of birth	Used for age validation
PAN number	Applicant's tax identification number
Employment type	Salaried, Self-employed, or Contract
Employer name	Name of employer or business
Monthly income	Applicant's monthly income
Existing monthly EMI	Existing monthly debt obligation
Requested loan amount	Amount requested by the applicant
Approved loan amount	Amount approved by the loan officer
Interest rate	Annual interest rate
Tenure in months	Loan repayment duration
Credit score	Applicant's credit score

Field	Purpose
Loan purpose	Reason for requesting the loan
Application date	Date of submission
Decision date	Date of approval or rejection
Disbursement date	Date when money was released
Status	Current application state
Rejection reason	Reason for rejection
Processing fee	Loan processing charge
Monthly EMI	Calculated monthly repayment amount
Created date	Record creation timestamp
Updated date	Last update timestamp

No separate customer, employer, payment, document, or loan entity should be created.

4. Application-number validation

Each loan application must have a unique application number.

Required format:

PL-YEAR-SEQUENCE

Examples:

PL-2026-00001
 PL-2026-00025
 PL-2027-00001

Rules:

- The application number must begin with **PL**.
- The year must contain exactly four digits.
- The year must match the application date year.
- The sequence must contain exactly five digits.
- Spaces are not allowed.
- The application number must be unique.

- The application number cannot be changed after creation.
- Lowercase input may be converted to uppercase before validation.

Invalid examples:

```
P-2026-00001
PL-26-00001
PL-2026-1
PL 2026 00001
HOME-2026-00001
```

5. Applicant-name validation

The applicant's full name must follow these rules:

- Name is mandatory.
- It must contain at least three characters.
- It must not exceed one hundred characters.
- It must contain at least two separate name parts.
- Numbers are not allowed.
- Unsupported special characters are not allowed.
- Apostrophes, periods, hyphens, and spaces may be allowed.
- Leading and trailing spaces must be removed.
- Multiple consecutive spaces must be converted into a single space.

Valid examples:

```
Rahul Sharma
Mary Anne Thomas
Anne-Marie Joseph
A. P. J. Kumar
```

Invalid examples:

```
Rahul123
R
John@
SingleName
```

6. Email validation

Rules:

- Email is mandatory.
- It must follow a valid email format.
- Spaces are not allowed.
- The domain must contain at least one period.
- The email must not start or end with a period.
- Consecutive periods are not allowed.
- Email comparison must be case-insensitive.
- The same email cannot have more than three active loan applications.
- The email should be converted to lowercase before storage.

Valid examples:

```
rahul.sharma@example.com  
user25@company.co.in
```

Invalid examples:

```
rahul@  
rahul example.com  
.rahul@example.com  
rahul..sharma@example.com
```

7. Phone-number validation

Rules:

- Phone number is mandatory.
- It must contain exactly ten digits.
- It must begin with 6, 7, 8, or 9.
- Spaces and country codes should be removed before validation when supported.
- Alphabetic characters are not allowed.
- Repeated values such as 9999999999 must be rejected.
- Sequential values such as 1234567890 should be rejected.
- The same phone number cannot have more than three active applications.

Valid examples:

9876543210
8123456789

Invalid examples:

1234567890
5555555555
98765
98AB543210

8. Date-of-birth and age validation

The applicant must satisfy age requirements.

Rules:

- Date of birth is mandatory.
- Date of birth cannot be in the future.
- Applicant must be at least 21 years old on the application date.
- Applicant must not be older than 60 years on the application date.
- The applicant's age at the end of the loan tenure must not exceed 65 years.
- Leap-year dates must be handled correctly.
- Age must be calculated using complete years, not by subtracting year values alone.

Example:

An applicant aged 59 requesting a seven-year loan must be rejected because the applicant would be older than 65 when the loan ends.

9. PAN-number validation

The PAN number must follow the standard format:

AAAAA9999A

Rules:

- PAN is mandatory.
- It must contain exactly ten characters.

- The first five characters must be uppercase letters.
- The next four characters must be digits.
- The final character must be an uppercase letter.
- Spaces are not allowed.
- Lowercase input may be converted to uppercase.
- The PAN must be unique for active applications.
- A PAN with an already approved or disbursed active loan must not be allowed to create another application unless company policy permits it.

Valid example:

ABCDE1234F

Invalid examples:

abcde1234f
ABCD1234F
ABCDE12345
ABCDE12A4F

10. Employment-type validation

Permitted values:

SALARIED
SELF_EMPLOYED
CONTRACT

Rules:

- Employment type is mandatory.
- Unsupported values must be rejected.
- Employer name is mandatory for salaried and contract applicants.
- Business name or professional description is mandatory for self-employed applicants.
- Contract applicants must have at least twelve months remaining in their contract period.
- Salaried applicants must have at least six months of continuous employment.
- Self-employed applicants must have operated their business for at least two years.

Because only one entity is allowed, employment duration and contract-end information may be stored directly inside `LoanApplication` when required.

11. Employer-name validation

Rules:

- Employer name is mandatory for salaried and contract applicants.
 - It must contain between two and one hundred characters.
 - It must not consist only of numbers.
 - Leading and trailing spaces must be removed.
 - Common business characters such as `&`, `.`, `-`, and parentheses may be permitted.
 - Placeholder values such as `NA`, `NONE`, or `UNKNOWN` must not be accepted when employer information is required.
-

12. Monthly-income validation

Rules:

- Monthly income is mandatory.
- It must be greater than zero.
- It must contain no more than two decimal places.
- Salaried applicants must have a minimum monthly income of ₹25,000.
- Self-employed applicants must have a minimum monthly income of ₹40,000.
- Contract applicants must have a minimum monthly income of ₹30,000.
- Monthly income must not exceed a configured upper limit such as ₹10,000,000.
- Negative income is invalid.
- Zero income is invalid.

Examples:

```
Salaried applicant with ₹20,000 income → reject  
Self-employed applicant with ₹35,000 income → reject  
Contract applicant with ₹45,000 income → accept income validation
```

13. Existing-EMI validation

Rules:

- Existing monthly EMI cannot be negative.
- It may be zero.
- It cannot exceed the applicant's monthly income.
- Existing EMI must contain no more than two decimal places.
- Existing EMI must be considered while calculating debt eligibility.

- An applicant whose existing EMI already exceeds the permitted debt ratio must be rejected.
-

14. Requested-loan-amount validation

Rules:

- Requested amount is mandatory.
- It must be greater than zero.
- Minimum personal loan amount is ₹50,000.
- Maximum personal loan amount is ₹2,500,000.
- Amount must be in multiples of ₹1,000.
- It must not contain more than two decimal places.
- The requested amount must not exceed twenty times the applicant's monthly income.
- Contract applicants cannot request more than twelve times their monthly income.
- Applicants with a credit score below 700 cannot request more than ten times their monthly income.

Example:

```
Monthly income: ₹50,000
Maximum under 20-times rule: ₹1,000,000
Requested amount: ₹1,200,000
Result: Reject
```

15. Credit-score validation

Rules:

- Credit score is mandatory.
 - Permitted range is 300 to 900.
 - Values outside this range must be rejected.
 - Applications with a score below 600 must be automatically rejected.
 - Scores from 600 to 649 require enhanced review.
 - Scores from 650 to 699 may be considered with a reduced loan amount.
 - Scores from 700 to 749 are normal-risk applications.
 - Scores of 750 or higher may qualify for preferred interest rates.
 - Credit score cannot be modified after the application has been approved or rejected without resetting the application to a review state.
-

16. Debt-to-income validation

The system must calculate a debt-to-income ratio.

Concept:

```
Debt-to-income ratio =  
Existing monthly EMI ÷ Monthly income × 100
```

Rules:

- The current debt-to-income ratio must not exceed 50%.
- After including the proposed loan EMI, the total debt-to-income ratio must not exceed 60%.
- For credit scores below 700, total debt-to-income must not exceed 50%.
- For contract employees, total debt-to-income must not exceed 45%.
- Division by zero must never occur because zero income must already be rejected.

Example:

```
Monthly income: ₹100,000  
Existing EMI: ₹30,000  
Proposed EMI: ₹25,000
```

```
Total EMI: ₹55,000  
Debt-to-income ratio: 55%
```

Result:

Accept for a normal salaried applicant with a sufficient score,
but reject for a contract applicant.

17. Loan-tenure validation

Permitted tenure range:

```
12 to 84 months
```

Rules:

- Tenure is mandatory.
- Tenure must be expressed as a whole number of months.

- Minimum tenure is twelve months.
- Maximum tenure is eighty-four months.
- Tenure must be in multiples of six months.
- Contract applicants cannot receive a tenure longer than their remaining contract period.
- The applicant's age at the end of the tenure must not exceed 65 years.
- Applications below a credit score of 650 cannot receive more than thirty-six months.
- Applications above ₹1,500,000 require a minimum tenure of twenty-four months.

Valid examples:

12 months
24 months
36 months
60 months
84 months

Invalid examples:

10 months
17 months
90 months
24.5 months

18. Interest-rate validation

Interest rate should be determined from business rules.

Example rate policy:

Credit score	Annual interest rate
750–900	10.50%
700–749	12.00%
650–699	14.50%
600–649	18.00%
Below 600	Not eligible

Additional rules:

- Self-employed applicants may receive an additional 1%.

- Contract applicants may receive an additional 1.5%.
 - Applications with a debt-to-income ratio above 50% may receive an additional 0.5%.
 - Interest rate cannot be manually set below the policy rate.
 - Interest rate must be between 5% and 30%.
 - It must contain no more than two decimal places.
 - Once the loan is disbursed, the interest rate cannot be changed.
-

19. Processing-fee validation

Processing fee may be calculated as:

2% of approved loan amount

Rules:

- Minimum processing fee is ₹1,000.
 - Maximum processing fee is ₹25,000.
 - Fee cannot be negative.
 - Fee must contain no more than two decimal places.
 - The fee must be calculated only after an approved loan amount is available.
 - Rejected applications must have a zero processing fee.
 - Disbursed applications must have a finalized processing fee.
-

20. EMI calculation validation

The monthly EMI must be calculated using:

- Approved loan amount
- Annual interest rate
- Loan tenure

Rules:

- EMI must not be entered manually.
- EMI must be calculated by the service layer.
- EMI must have two decimal places.
- EMI cannot be zero for approved or disbursed loans.
- EMI must be zero or absent for new and rejected applications.
- EMI must be recalculated when approved amount, tenure, or interest rate changes before disbursement.
- EMI cannot be recalculated after disbursement unless the loan is formally restructured.

The project should document the EMI formula even though no code is required in the case study.

21. Loan-purpose validation

Permitted loan purposes may include:

```
MEDICAL
EDUCATION
HOME_RENOVATION
WEDDING
TRAVEL
DEBT_CONSOLIDATION
EMERGENCY
OTHER
```

Rules:

- Loan purpose is mandatory.
 - Unsupported values must be rejected.
 - When `OTHER` is selected, a purpose description must be provided.
 - Purpose description must contain between ten and two hundred characters.
 - Illegal, gambling-related, or prohibited purposes must be rejected according to company policy.
 - A high loan amount for a minor purpose may be marked for manual review.
-

22. Application-date validation

Rules:

- Application date is mandatory.
 - It cannot be in the future.
 - It cannot be more than thirty days in the past during creation.
 - Once created, the application date cannot be changed.
 - Application number year must match the application date year.
 - Created date must not be later than the current system date.
-

23. Application-status values

Permitted statuses:

DRAFT
SUBMITTED
UNDER_REVIEW
APPROVED
REJECTED
DISBURSED
CLOSED
CANCELLED

Each status represents a specific stage.

DRAFT

The application is incomplete and has not been formally submitted.

SUBMITTED

All mandatory applicant information has been supplied.

UNDER_REVIEW

The application is being assessed.

APPROVED

The loan has been approved but not yet disbursed.

REJECTED

The loan application has been rejected.

DISBURSED

The approved amount has been released.

CLOSED

The loan process has been completed or fully settled.

CANCELLED

The applicant cancelled the request before disbursement.

24. Status-transition validation

The system must not allow arbitrary status changes.

Allowed transitions:

```
DRAFT → SUBMITTED
DRAFT → CANCELLED

SUBMITTED → UNDER_REVIEW
SUBMITTED → CANCELLED

UNDER_REVIEW → APPROVED
UNDER_REVIEW → REJECTED
UNDER_REVIEW → CANCELLED

APPROVED → DISBURSED
APPROVED → CANCELLED

DISBURSED → CLOSED
```

Disallowed examples:

```
DRAFT → APPROVED
DRAFT → DISBURSED
REJECTED → APPROVED
CANCELLED → DISBURSED
CLOSED → UNDER_REVIEW
DISBURSED → CANCELLED
```

Rules:

- A rejected application cannot be approved directly.
- A cancelled application cannot be reactivated.
- A closed application cannot be modified.
- A loan cannot be disbursed unless its current status is approved.
- A disbursed loan cannot be deleted.
- A status change must also validate the fields required for the target status.

25. Submission validation

Before moving from **DRAFT** to **SUBMITTED**, verify:

- Applicant name is valid.
- Email is valid.
- Phone number is valid.
- Date of birth is valid.
- PAN is valid.
- Employment type is valid.
- Employer or business information is present.
- Income is valid.
- Requested amount is valid.
- Tenure is valid.
- Loan purpose is valid.
- Credit score is available.
- Existing EMI is valid.

An incomplete application must remain in **DRAFT**.

26. Approval validation

Before moving to **APPROVED**, verify:

- Current status is **UNDER_REVIEW**.
 - Credit score is at least 600.
 - Applicant satisfies the age policy.
 - Income satisfies employment-type requirements.
 - Requested amount is within the eligible amount.
 - Debt-to-income ratio is within limits.
 - Approved amount has been entered.
 - Approved amount is greater than zero.
 - Approved amount does not exceed requested amount.
 - Approved amount does not exceed calculated eligibility.
 - Interest rate has been calculated.
 - Processing fee has been calculated.
 - Monthly EMI has been calculated.
 - Decision date has been supplied.
 - Rejection reason is empty.
-

27. Rejection validation

Before moving to **REJECTED**, verify:

- Current status is **UNDER_REVIEW**.
- Rejection reason is mandatory.
- Rejection reason must contain between ten and five hundred characters.
- Decision date is mandatory.
- Approved amount must be zero or absent.
- Monthly EMI must be zero or absent.
- Processing fee must be zero.
- Disbursement date must be absent.

Suggested rejection reasons include:

- Low credit score
 - Insufficient income
 - Excessive debt obligations
 - Applicant age outside permitted limits
 - Requested amount exceeds eligibility
 - Invalid identification information
 - Employment conditions not satisfied
-

28. Disbursement validation

Before moving from **APPROVED** to **DISBURSED**, verify:

- Current status is **APPROVED**.
- Approved amount is greater than zero.
- Interest rate is present.
- Tenure is valid.
- EMI is present and greater than zero.
- Processing fee is finalized.
- Decision date is present.
- Disbursement date is mandatory.
- Disbursement date cannot be before the approval date.
- Disbursement date cannot be in the future.
- Rejection reason must be absent.
- The applicant must not have another disbursed active loan when company policy disallows it.

After disbursement, the following fields become immutable:

- Application number
- PAN number
- Approved loan amount

- Interest rate
 - Tenure
 - EMI
 - Application date
 - Decision date
 - Disbursement date
-

29. Cancellation validation

Rules:

- A draft, submitted, under-review, or approved application may be cancelled.
- A rejected application cannot be cancelled.
- A disbursed application cannot be cancelled.
- A closed application cannot be cancelled.
- A cancellation reason should be mandatory.
- Approved financial values may be retained for audit but must not be treated as active.

Because only one entity is used, the cancellation reason may use the same general decision-reason field or an additional field inside `LoanApplication`.

30. Approved-amount validation

Rules:

- Approved amount is mandatory only for approved and disbursed applications.
 - It must be greater than zero.
 - It must not exceed the requested amount.
 - It must not exceed policy eligibility.
 - It must be in multiples of ₹1,000.
 - It must not exceed ₹2,500,000.
 - It must contain no more than two decimal places.
 - Rejected, draft, submitted, and cancelled applications should not have a positive approved amount.
-

31. Decision-date validation

Rules:

- Decision date is mandatory for approved and rejected applications.
- It must not be before the application date.
- It cannot be in the future.

- It must be absent for draft and submitted applications.
 - It cannot be modified after disbursement.
 - Approval or rejection must occur within thirty days of submission.
 - Applications not decided within thirty days may be flagged as expired or overdue.
-

32. Duplicate-application validation

An application should be considered a possible duplicate when:

- The same PAN has an application in `SUBMITTED`, `UNDER_REVIEW`, `APPROVED`, or `DISBURSED`.
- The same email and phone number are used with a different PAN.
- The same applicant submits the same requested amount within a short period.
- Multiple active applications exist for the same PAN.

Rules:

- Exact active duplicates must be rejected.
 - Draft duplicates may be returned to the applicant for completion instead of creating a new record.
 - Rejected applications may be resubmitted only after a configured waiting period, such as ninety days.
 - A new application after rejection must receive a new application number.
-

33. Update rules

Different fields may be updated depending on status.

In `DRAFT`

Most applicant and loan-request fields may be updated.

In `SUBMITTED`

Only limited corrections should be allowed, such as:

- Phone number
- Email
- Employer name

Major financial changes should return the application to `DRAFT`.

In **UNDER_REVIEW**

Loan officers may update:

- Credit score
- Approved amount
- Interest rate
- Decision information
- Review-related details

In **APPROVED**

Only disbursement-related details may be added.

In **REJECTED**, **DISBURSED**, **CLOSED**, or **CANCELLED**

Business information should be treated as immutable.

34. Delete rules

The application must support deletion, but deletion must be restricted.

An application may be deleted only when:

- Its status is **DRAFT**, or
- Its status is **CANCELLED** and it has never been approved or disbursed.

An application must not be deleted when its status is:

SUBMITTED
UNDER_REVIEW
APPROVED
REJECTED
DISBURSED
CLOSED

This ensures that applications with financial or decision history remain available for audit.

Deletion should return a clear result:

- Deleted successfully
- Application not found
- Deletion not permitted for the current status

35. Search requirements

Although only one entity is used, the system should support several search operations.

Search by:

- Database ID
- Application number
- PAN number
- Email
- Phone number
- Status
- Employment type
- Application-date range
- Requested-amount range
- Credit-score range

Combined searches may include:

```
All UNDER_REVIEW applications submitted this month
All APPROVED applications above ₹1,000,000
All applications with credit score below 650
All DISBURSED loans for self-employed applicants
```

Search input must also be validated.

Examples:

- Start date cannot be after end date.
- Minimum amount cannot exceed maximum amount.
- Credit-score limits must remain between 300 and 900.
- Unsupported status values must be rejected.

36. Database constraints

The PostgreSQL table should include suitable constraints.

Recommended constraints:

- Primary key on ID
- Unique constraint on application number
- Valid PAN format

- Non-negative monthly income
- Non-negative existing EMI
- Positive requested loan amount
- Credit score between 300 and 900
- Tenure between 12 and 84
- Non-negative approved amount
- Non-negative processing fee
- Non-negative EMI
- Valid permitted status
- Unique active-application logic where feasible
- Application date not later than decision date
- Decision date not later than disbursement date

Complex business rules should remain in the service layer, while fundamental data-integrity rules should also be enforced in PostgreSQL.

37. Layer responsibilities

Model layer

The `LoanApplication` class represents the application data.

Lombok may generate:

- Getters
- Setters
- Constructors
- Builder methods
- `toString`
- `equals`
- `hashCode`

Configuration layer

Responsible for:

- Loading database properties
- Creating the PostgreSQL `DataSource`
- Enabling component scanning
- Registering Spring-managed beans

Repository layer

Responsible for:

- Standard JDBC operations
- Opening connections
- Creating prepared statements
- Executing SQL
- Mapping result sets
- Closing JDBC resources
- Converting SQL exceptions into application exceptions

Service layer

Responsible for:

- Business validation
- Eligibility calculations
- Status-transition validation
- Duplicate checks
- EMI calculation
- Interest-rate calculation
- Processing-fee calculation
- Calling repository operations

Main application layer

Responsible for:

- Starting the Spring context
- Obtaining the service bean
- Demonstrating use cases
- Displaying operation results

38. Required repository operations

The repository should conceptually support:

1. Create a loan application.
2. Find an application by ID.
3. Find an application by application number.
4. Find active applications by PAN.
5. Find applications by status.
6. Retrieve all applications.

7. Update an application.
8. Update only the application status.
9. Delete an eligible application.
10. Check whether an application number exists.
11. Check whether an active application exists for a PAN.
12. Search by date, amount, credit score, or employment type.

All SQL must use prepared statements.

39. Required service operations

The service should conceptually support:

1. Create a draft application.
 2. Submit an application.
 3. Retrieve an application.
 4. Retrieve all applications.
 5. Update draft details.
 6. Move an application to review.
 7. Evaluate eligibility.
 8. Approve an application.
 9. Reject an application.
 10. Cancel an application.
 11. Disburse an approved loan.
 12. Close a disbursed loan.
 13. Delete an eligible draft.
 14. Search applications.
 15. Calculate EMI.
 16. Calculate interest rate.
 17. Calculate processing fee.
 18. Calculate debt-to-income ratio.
-

40. Properties-file requirements

Database configuration must be placed in a properties file.

Suggested properties:

```
Database host  
Database port  
Database name
```



```
Database username
Database password
```

Optional business properties may also be stored in a separate configuration file:

```
Minimum applicant age
Maximum applicant age
Maximum age at tenure completion
Minimum loan amount
Maximum loan amount
Minimum credit score
Maximum tenure
Minimum income by employment type
Maximum debt-to-income ratio
Processing-fee percentage
```

Using configurable values avoids hard-coding all policies inside business logic.

41. Validation execution order

When creating or submitting a loan application, validations should occur in the following order:

1. Check that the application object is not null.
2. Normalize string values.
3. Validate applicant name.
4. Validate email.
5. Validate phone number.
6. Validate date of birth.
7. Calculate applicant age.
8. Validate PAN.
9. Validate employment type.
10. Validate employer or business information.
11. Validate monthly income.
12. Validate existing EMI.
13. Validate requested amount.
14. Validate tenure.
15. Validate loan purpose.
16. Validate credit score.
17. Check for duplicate applications.
18. Calculate debt-to-income ratio.
19. Calculate eligible loan amount.
20. Determine interest rate.
21. Calculate estimated EMI.
22. Validate age at tenure completion.

23. Validate the initial status.

24. Store the application.

The order is important because later calculations depend on earlier validated values.

42. Exception categories

The project should define clear exception categories.

Validation exception

Used for invalid applicant or loan data.

Examples:

- Invalid PAN
- Invalid email
- Applicant is underage
- Requested amount exceeds policy

Application-not-found exception

Used when the requested application does not exist.

Duplicate-application exception

Used when an active application already exists.

Invalid-status-transition exception

Used when the requested status change is not permitted.

Loan-ineligible exception

Used when the applicant fails financial eligibility rules.

Repository exception

Used when a JDBC or PostgreSQL operation fails.

Error messages should explain exactly which business rule failed.

43. Suggested test scenarios

Successful scenarios

1. Create a valid draft application.
2. Submit a completed application.
3. Move a submitted application to review.
4. Approve an eligible salaried applicant.
5. Approve an eligible self-employed applicant.
6. Reject an applicant with a low credit score.
7. Disburse an approved loan.
8. Close a disbursed loan.
9. Cancel a draft application.
10. Delete a draft application.
11. Retrieve applications by status.
12. Retrieve an application by application number.

Applicant-validation failures

1. Applicant name contains numbers.
2. Email format is invalid.
3. Phone number has fewer than ten digits.
4. Phone number contains all repeated digits.
5. Applicant is younger than 21.
6. Applicant is older than 60.
7. Invalid PAN format.
8. Employment type is unsupported.
9. Employer name is missing.
10. Monthly income is zero.

Loan-validation failures

1. Requested amount is below ₹50,000.
2. Requested amount exceeds ₹2,500,000.
3. Requested amount is not a multiple of ₹1,000.
4. Requested amount exceeds income eligibility.
5. Credit score is below 600.
6. Tenure is below twelve months.
7. Tenure is not a multiple of six.
8. Tenure causes applicant age to exceed 65.
9. Existing EMI exceeds monthly income.
10. Total debt-to-income ratio exceeds the limit.

Status-validation failures

1. Attempt to approve a draft.

2. Attempt to disburse a submitted application.
3. Attempt to approve a rejected application.
4. Attempt to cancel a disbursed loan.
5. Attempt to delete an approved application.
6. Reject an application without a reason.
7. Approve an application without an approved amount.
8. Disburse without a disbursement date.
9. Set a decision date before the application date.
10. Set a disbursement date before the approval date.

Duplicate-validation failures

1. Same application number is used twice.
 2. Same PAN has another active application.
 3. Same PAN attempts to apply again before the waiting period.
 4. Same email and phone are used with suspiciously different identity information.
-

44. End-to-end demonstration flow

A complete demonstration should follow these steps:

1. Start PostgreSQL.
2. Create the application database.
3. Configure database properties.
4. Start the Spring IoC container.
5. Load configuration values.
6. Create the `DataSource`.
7. Create or verify the loan-application table.
8. Create a valid draft application.
9. Retrieve and display the draft.
10. Update missing information.
11. Submit the application.
12. Move it to review.
13. Calculate eligibility.
14. Calculate the interest rate.
15. Calculate EMI.
16. Approve the application.
17. Retrieve the approved application.
18. Disburse the loan.
19. Attempt an invalid update after disbursement.
20. Confirm that the update is rejected.
21. Close the loan.
22. Retrieve the final record.
23. Attempt to delete it.
24. Confirm that deletion is rejected.

25. Create another invalid application.
 26. Display the validation errors.
-

45. Expected learning outcomes

After completing the project, the learner should understand:

- Spring IoC and dependency injection
 - Component scanning
 - Java-based Spring configuration
 - Loading values from properties files
 - Standard JDBC programming
 - PostgreSQL connectivity
 - Prepared statements
 - Result-set mapping
 - Try-with-resources
 - Repository and service separation
 - Lombok usage
 - Complex business validation
 - Status-transition rules
 - Database constraints
 - Exception handling
 - Financial calculations
 - Single-entity application design
-

46. Final assignment statement

Develop a console-based Personal Loan Application Processing System using plain Spring Framework, standard JDBC, PostgreSQL, Lombok, Maven, and properties-based configuration.

Use only the `LoanApplication` entity.

The application must implement complete CRUD operations, application searching, financial eligibility checks, EMI calculations, duplicate detection, complex field validations, controlled status transitions, and restricted deletion.

Do not use Spring Boot, Spring Data JPA, Hibernate, JPA, `JdbcTemplate`, REST APIs, or additional business entities.